



Systematic API Testing Through Model Checking and Executable Contracts

Ana Catarina Ribeiro
NOVA University Lisbon
Caparica, Portugal
acm.ribeiro@campus.fct.unl.pt

Margarida Mamede
NOVA University Lisbon
Caparica, Portugal
mm@fct.unl.pt

Carla Ferreira
NOVA University Lisbon
Caparica, Portugal
carla.ferreira@fct.unl.pt

Abstract—Automated black-box testing of APIs typically relies on interface specifications that define available operations and data schemas, but offer limited or no behavioural semantics. This semantic gap amplifies the test-oracle problem and limits the generation of effective, stateful call sequences.

We introduce ICEPICK, a framework that achieves systematic state-space coverage for API testing by leveraging model checking. ICEPICK uses TLA^+ to formally model API state evolution, employs the TLC model checker to exhaustively explore reachable states, and generates test sequences that provably cover the behavioural model. To mitigate state-space explosion and improve sequence extraction, we introduce a coverage-guided breadth-first traversal of the TLC state-space graph. To address oracle limitations beyond HTTP status codes, we propose GLACIER, a first-order logic contract language that enriches API specifications with executable semantic contracts, enabling automated behavioural verification during test execution.

We evaluate ICEPICK on EvoMaster Benchmark systems, demonstrating that model-checking-guided exploration achieves complete state coverage and reveals faults in multi-operation interactions. We also analyse scalability to characterise practical limits and applicability requirements. Overall, ICEPICK provides reproducible test suites with strong coverage guarantees for critical API-based systems.

Index Terms—model checking, black-box testing, API testing, automated testing, TLA^+ , TLC, OAS, REST, RESTful API.

I. INTRODUCTION

Model checking [1] and testing are complementary approaches to establishing system correctness. Model checking provides formal guarantees by exhaustively exploring a model's complete state-space to verify specific properties. Testing, by contrast, validates the actual implementation by executing it with diverse inputs to uncover edge cases [2], [3].

In this paper, we integrate both approaches by leveraging model checking's systematic state-space exploration to generate test cases that comprehensively cover a program's behaviour in a black-box setting. We apply this combined approach to microservice architectures, which decompose business logic into a suite of small services [4]. Because such systems are often accessed by heterogeneous third-party clients, their implementations are typically hidden, making them opaque to users. Most microservice implementations follow REST, a set of architectural principles for building web services over HTTP [5], and their APIs are commonly referred to as RESTful APIs. While we focus on microservice

architectures, the approach applies to other API-based systems that are testable through their interfaces.

Testing microservices poses unique challenges, as the main artefact is the API definition, typically specified using the OpenAPI Specification (OAS) [6]. While OAS is widely accepted as a standard, it lacks detail to fully capture the system behaviour. These specifications outline operation endpoints, expected data types for each HTTP interaction, and response codes for various scenarios, yet they fall short of capturing the service's underlying logic and dynamic behaviours.

The widespread adoption of RESTful APIs has driven demand for automated black-box testing tools. Existing tools typically derive tests exclusively from API specifications and face several limitations despite promising results [7]–[15]. Specifications are often incomplete (missing parameters or dependencies), inconsistent (mismatched names across operations), outdated (not synchronised with implementation), or non-compliant with RESTful guidelines. However, the most pervasive challenge is the oracle problem [16]. State-of-the-art tools often treat HTTP status codes as failure oracles, but this is unreliable. For instance, a 5xx response indicates an exception, which may be caused by many factors rather than a violation of specified requirements.

To address these challenges, we introduce ICEPICK, a framework for black-box testing of RESTful APIs that uses TLA^+ [17], a formal language for specifying and designing complex software systems [18], together with its model checker, TLC [19], [20]. We leverage TLC's State Space Graph (SSG) to systematically generate test suites covering modelled behaviour. To address the oracle problem, we propose GLACIER, a contract specification language for RESTful APIs based on first-order logic. GLACIER enables developers to enrich API specifications with practical properties for test generation, transforming specifications into valuable testing artefacts. Beyond providing useful test oracles, GLACIER assigns precise semantics to the API and adapts to any extensible RESTful API specification language.

Recognising that writing formal specifications imposes additional development overhead, we provide a tool that automatically generates GLACIER contracts from OAS files using HTTP semantics and RESTful conventions. Additionally, we offer a catalogue [21] of standard formulae definitions for common properties such as referential integrity constraints.

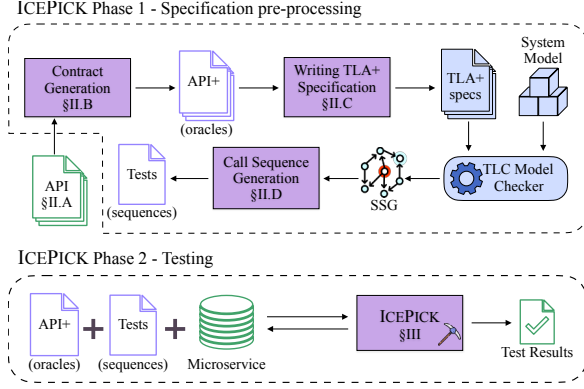


Fig. 1. ICEPICK framework overview.

As noted, while we illustrate the technique using microservice architectures and RESTful APIs, it should also apply to systems with a well-defined programmatic interface.

Following an overview of our solution, we introduce key background concepts and survey related work on the automated testing of RESTful APIs and on model-checking-based test generation. We then present the paper’s contributions:

- **GLACIER** [21]: a contract specification language for RESTful APIs based on first-order logic and REST semantics, with an automated contract generator and a catalogue of reusable formulae for common API properties.
- **ICEPICK framework**: a model checking-based approach for generating comprehensive test suites from TLA⁺ specifications and GLACIER contracts.
- **Empirical evaluation** using the EvoMaster Benchmark (EMB) [22].
- **Replication package** [23] with all source-code and specifications.

II. OVERVIEW

This section overviews the ICEPICK framework using a tournament management system. The Tournaments Application comprises three APIs, Tournaments, Players, and Enrolments, enabling creation, update, and removal of these resources. ICEPICK’s workflow has two phases: specification preprocessing and systematic testing, illustrated in Figure 1.

In the first phase, we generate operation-specific contracts from API specifications and combine them into an extended specification to produce a TLA⁺ service abstraction. We then manually design the system model and check it with TLC to obtain the SSG. In the second phase, we traverse the SSG to generate call sequences, execute them against the system under test, and check observed behaviour against the specification using automatically generated test data [24]. The process concludes with a detailed test report supporting reproducibility.

A. OpenAPI Specification

The OAS describes RESTful APIs at the interface level: endpoints, input/output schemas, and status codes. While widely adopted, it neither models system state nor captures

behavioural semantics, though it is extensible. In ICEPICK, we exploit this extensibility to attach operation-level contracts and invariants supporting test generation and oracle checking.

Listing 1 shows the OAS description for postPlayer in the Players API, detailing its endpoint (line 1), HTTP verb (post), a human-readable summary, unique identifier (line 4), request-body schema reference (line 5), and a successful response definition (lines 11–17). This information supports the generation of syntactically valid calls but is insufficient to determine whether the observed behaviour is semantically correct, motivating the contracts in §II-B.

```

1  /players:
2    post:
3      summary: Add a new player.
4      operationID: postPlayer
5      requestBody:
6        content:
7          application/json:
8            schema:
9              $ref: #/components/schemas/Player
10     required: true
11     responses:
12       200:
13         description: Successful insertion.
14         content:
15           application/json:
16             schema:
17               $ref: #/components/schemas/Player

```

Listing 1. OAS description of postPlayer in the Tournament Application.

B. Contract Generation

This module generates operation-level contracts (pre- and postconditions) from the OAS using CRUD (Create, Read, Update, Delete) semantics, expressed in a domain-specific language called GLACIER. The generator produces an extended specification that retains the original OAS structure while incorporating contracts and invariants, which serve as executable oracles during testing.

This follows the established principle of using contracts as verification artefacts [25]–[28], instantiated here for RESTful APIs via CRUD-inferred pre- and postconditions evaluated during black-box execution.

Listing 2 shows the inferred contract for postPlayer. The requires clause captures the expected precondition that the player does not yet exist (404), while the ensures clause captures the expected postcondition that the player exists after insertion (200). The @ symbol is a reference to “this” operation, enabling contracts to access the current request and response.

```

1  /players:
2    POST:
3      operationID: postPlayer
4      requires:
5        - res_code(GET /players/req_body(@){pid}) = 404
6      ensures:
7        - res_code(GET /players/req_body(@){pid}) = 200
8        - req_body(@) = res_body(@)

```

Listing 2. postPlayer operation contract.

Generated contracts are sound and automatically inferred, but intentionally partial: they may omit service-specific semantics. A manually added predicate can express application-specific consistency constraints (e.g., referential integrity)

between tournament enrolments and the players registered in each tournament, which are not generally inferable from CRUD semantics alone. To support manual completion, we provide a catalogue of reusable GLACIER predicates for common API properties [21]. For instance, `postPlayer` can be strengthened by adding the predicate on line 8 of Listing 2, which validates that the returned payload matches the submitted input – a semantic check derivable from the OAS definition that the operation receives and returns a player resource.

In contrast, `deletePlayer` returns the player on success but does not include a player resource as input. Instead, it takes the player identifier as a path parameter. Consequently, the same equality predicate cannot be evaluated directly from the operation input. To address this, GLACIER provides a `prev` operator that allows predicates to refer to values obtained before the operation executes. Listing 3 shows the inferred contract for `deletePlayer`, using `prev` to compare the deleted resource with the value observed prior to deletion.

```
1 /players/{pid}:
2 DELETE:
3   operationID: deletePlayer
4   requires:
5     - res_code(GET /players/{pid}) = 200
6   ensures:
7     - res_code(GET /players/{pid}) = 404
8     - req_body(@) = prev(res_body(GET /players/{pid}))
```

Listing 3. `deletePlayer` operation contract.

Finally, GLACIER supports quantification over collections, enabling expressive invariants. Listing 4 shows an invariant stating that, for every tournament, the number of enrolled players never exceeds the tournament capacity.

```
1 invariants:
2 - for t in res_body(GET /tournaments) :-
3   res_body(GET /tournaments/{t.tid}/players).len ≤
4   res_body(GET /tournaments/{t.tid}/capacity)
```

Listing 4. Invariant for the Tournaments API.

The previous examples introduce GLACIER’s main constructs, while Listing 5 summarises its grammar. A formula is either a quantified statement over a collection or a boolean expression. Quantified formulas support both universal and existential quantification over multiple variables. Boolean expressions combine comparisons with standard logical operators, including implication ($\Rightarrow$). The built-in functions `req_body`, `res_body`, and `res_code` apply to the current operation (`@`) or an explicit HTTP request. The `prev` operator captures a value before execution, allowing postconditions to refer to the prior state. An optional suffix selects a field or applies a function such as `len`. The full grammar appears in [21].

```
1 formula ::=
2   | ('for' | 'exists') vars (':-' | ':') formula
3   | expr (boolOp expr)*
4 vars ::= varID 'in' call (',' vars)?
5 expr ::=
6   | ('res_code' | 'res_body' | 'req_body') '(' '@' |
7     method url ')' suffix?
8   | 'prev' '(' call ')'
9   | expr comparator expr
10 suffix ::= '{' field '}' | '.' func
```

Listing 5. GLACIER grammar.

C. TLA⁺ Specification

TLA⁺ provides a mathematical foundation for specifying systems [18], combining temporal logic and set theory. Paired with TLC [19], [20], it supports exhaustive state-space exploration and automated verification of safety and liveness properties. In ICEPICK, we use TLA⁺ to define a conservative abstraction that retains only state-relevant information for call sequence generation and oracle checking of a given system.

A TLA⁺ specification comprises three core elements: constants that define finite domains, variables that represent the abstract system state, and actions that describe state transitions [29]. In our example, each resource type is represented by a finite set of identifiers, and the system state is captured by maps from identifiers to simplified records. Operational contracts become action guards and next-state constraints, while global GLACIER invariants become TLC-checked state invariants. We introduce an auxiliary boolean variable `final` marking terminal states (those in which all operations in a valid test sequence have been exercised) to guide SSG traversal in §II-D. The abstraction retains exactly the information needed to decide whether operations are enabled, how they change abstract state, and whether the invariants used by ICEPICK can be evaluated.

States and Types: Listing 6 defines the constant domains (player, tournament, and enrolment identifiers, plus a finite set N of positive naturals for capacities) and the state variables (three resource maps and the terminal-state flag). Each constant is constrained by an `ASSUME` clause requiring non-empty identifier sets and a finite set N (not shown for brevity). Here, `EmptyMap` denotes the empty resource map, `IsMap` checks whether a variable is a map with the expected key and value domains, and `TS` identifies terminal states.

```
1 CONSTANTS PID, TID, EID, N
2 VARIABLES tournaments, players, enrolments, final
```

Listing 6. TLA⁺ constants and variables.

Resources are represented as records omitting schema fields irrelevant to reachability, as shown in Listing 7. The initial state sets all resource maps to empty and `final` to `FALSE` (no lifecycles have completed yet).

```
1 PlayerType == [ts: SUBSET TID]
2 TournamentType == [ps: SUBSET PID, c: N]
3 EnrolmentType == [pid: PID, tid: TID]
4
5 Init ==
6   ∧ players = EmptyMap
7   ∧ tournaments = EmptyMap
8   ∧ enrolments = EmptyMap
9   ∧ final = FALSE
```

Listing 7. TLA⁺ resource types and initial state.

Invariants: Type correctness is enforced by a type invariant that verifies that resource maps are keyed by their identifiers and that `final` is a boolean. Beyond type correctness, we translate GLACIER invariants into TLA⁺ predicates. Listing 8 shows the type invariant checked by TLC: all state variables are maps except `final`, which is a boolean.

```

1 TypeInv ==
2   ∧ IsMap(PID, PlayerType, players)
3   ∧ IsMap(TID, TournamentType, tournaments)
4   ∧ IsMap(EID, EnrolmentType, enrolments)
5   ∧ final ∈ BOOLEAN

```

Listing 8. Type invariant.

Listing 9 specifies the invariant that the number of enrolled players never exceeds the tournament capacity, which model checking verifies for all reachable states.

```

1 Inv1 == ∀ tid ∈ TID :
2   tid ∈ DOMAIN tournaments ⇒
3   |tournaments[tid].ps| ≤ tournaments[tid].c

```

Listing 9. Tournaments capacity invariant.

Operations as TLA⁺ actions: Each state-mutating API operation becomes a TLA⁺ action, with pre- and postconditions derived from GLACIER contracts. Listing 10 shows the complete action for postPlayer: the precondition requires that the player does not exist; the postcondition ensures it does after insertion. Line 6 inserts the new player record, line 8 updates the terminal-state flag, and line 9 specifies that tournaments and enrolments remain unchanged.

```

1 Requires(pid) == pid ∉ DOMAIN players
2 Ensures(pid) == pid ∈ DOMAIN players'
3
4 postPlayer(pid) ==
5   ∧ Requires(pid)
6   ∧ players' = MapPut(players, pid, [ts → ∅])
7   ∧ Ensures(pid)
8   ∧ final' = TS(players', tournaments, enrolments)
9   ∧ UNCHANGED ⟨tournaments, enrolments⟩

```

Listing 10. TLA⁺ postPlayer operation definition.

Remarks: Our TLA⁺ specification is lifecycle-oriented and intentionally abstract, focusing on operations that induce state transitions (post and delete). Read-only operations are not considered, and update operations (put and patch) that modify abstracted fields are handled at the call sequence level (§II-D).

D. Call Sequence Generation Algorithm

The SSG can be modelled as a directed graph $G = (V, E)$, where V is the set of states explored by TLC and E is the set of state transitions corresponding to API operations. It has a single initial state i , multiple final states, and a super-final sink f connecting all final states: for every final state u , $(u, f) \in E$. Moreover, every state lies on some path from i to f , and the graph has no parallel edges.

We developed a custom coverage-guided algorithm based on breadth-first search (BFS) [30] to generate call sequences, described by paths from i to f . We chose the BFS-based approach since it produces the shortest paths from the initial state to the super-final state, thereby minimising the length of generated call sequences and reducing the number of API requests required during testing. The set of all selected call sequences achieves transition coverage, in the sense that every transition belongs to some selected path. The algorithm operates in three stages: first, it collects paths from i , classifying them as complete (reaching f) or incomplete; then, it collects

a path from each state to f , used in the last step to complete incomplete paths.

To simplify notation, states are denoted by integers from i to f . The digraph $G = (V, E)$ is implemented with two arrays of adjacency-lists: $\text{outAdj}[u] = \{v \mid (u, v) \in E\}$ and $\text{inAdj}[v] = \{u \mid (u, v) \in E\}$, where $u, v \in V$.

Stage 1: Collecting paths from the initial state: Algorithm 1 starts by creating and initialising some structures: the sets of complete and incomplete paths and the FIFO queue start empty, no state has been found, and there is no path from i to any state (lines 2–4). The traversal starts at i : the zero-length path from i to i is stored, i is enqueued and is marked as found (lines 6–7). In the while loop, the algorithm dequeues a state and explores all of its outgoing transitions. For each transition, it extends a copy of the current path by appending the destination state (line 11). Then, three distinct cases can hold. If the destination state is f , since f has been declared found in line 5, the new path is added to the set of complete paths. Otherwise, either the destination state has not yet been found, in which case the new path will be extended further, the state is enqueued and marked as found, or the destination state has already been enqueued, and the new path is added to the set of incomplete paths. When the queue is empty, the sets of complete and incomplete paths are returned.

Algorithm 1 Collects paths from i (complete and incomplete).

```

input:
  i : initial state,   f : final state
  outAdj : states adjacent to each state by outgoing transitions
output:
  cmp : set of complete paths
  inc : set of incomplete paths
1: function PATHSTO
2:   cmp ← ∅,   inc ← ∅
3:   paths[i...f] ← ⊥
4:   fifo ← ∅,   found[i...f] ← ⊥
5:   found[f] ← T
6:   paths[i] ← (i)
7:   ENQUEUE(fifo, i),   found[i] ← T
8:   while fifo ≠ ∅ do
9:     u ← DEQUEUE(fifo)
10:    for all v ∈ outAdj[u] do
11:      pathV ← APPEND(CLONE(paths[u]), (v))
12:      if found[v] = ⊥ then
13:        paths[v] ← pathV
14:        ENQUEUE(fifo, v),   found[v] ← T
15:      else if v = f then
16:        cmp ← cmp ∪ {pathV}
17:      else
18:        inc ← inc ∪ {pathV}
19:   return cmp, inc

```

Let cmp and inc be the returned sets. Since every state lies on some path from i to f and each transition is analysed once, being integrated in some path, set $\text{cmp} \cup \text{inc}$ covers all transitions. Besides, the size of $\text{cmp} \cup \text{inc}$ is, at most, the number of transitions, as no transition is shared across paths.

Stage 2: Collecting paths to the super-final state: Algorithm 2 collects, for each state, a shortest path from that state to f . Like Algorithm 1, it also executes a breadth-first search,

but now the traversal starts at f , transitions are analysed from the destination to the origin state, and new paths are built by prefixing the origin to a copy of the current path (line 10).

Algorithm 2 Collects a path from each state to f .

```

input:
   $i$  : initial state,    $f$  : final state
   $\text{inAdj}$  : states adjacent to each state by incoming transitions
output:
   $\text{paths}$  : a path from each state to  $f$ 
1: function PATHSFROM
2:    $\text{paths}[i..f] \leftarrow \perp$ 
3:    $\text{fifo} \leftarrow \emptyset$ ,    $\text{found}[i..f] \leftarrow \perp$ 
4:    $\text{paths}[f] \leftarrow (f)$ 
5:   ENQUEUE( $\text{fifo}$ ,  $f$ ),    $\text{found}[f] \leftarrow \top$ 
6:   while  $\text{fifo} \neq \emptyset$  do
7:      $v \leftarrow \text{DEQUEUE}$ ( $\text{fifo}$ )
8:     for all  $u \in \text{inAdj}[v]$  do
9:       if  $\text{found}[u] = \perp$  then
10:         $\text{paths}[u] \leftarrow \text{APPEND}(u, \text{CLONE}(\text{paths}[v]))$ 
11:        ENQUEUE( $\text{fifo}$ ,  $u$ ),    $\text{found}[u] \leftarrow \top$ 
12:   return  $\text{paths}$ 

```

Stage 3: Selecting call sequences for diversity: Since the set $\text{cmp} \cup \text{inc}$ already covers all transitions, the main goal of Algorithm 3 is to extend the incomplete paths. The set of selected paths starts with cmp , which contains only complete paths (line 4). For each incomplete path collected in the first phase, $p = i u_1 u_2 \dots u_n v$ (for some $n \geq 1$), a new complete path is created by first deleting the last state from p and then appending the path from v to f , as computed in the second phase (lines 6–7). The new path is added to the selected paths.

Algorithm 3 Selects the call sequences from the SSG.

```

1: function SELECTSEQUENCES( $\text{ssg}$  : state-space graph)
2:   ( $\text{cmp}$ ,  $\text{inc}$ )  $\leftarrow \text{PATHSTO}(\text{ssg}.i, \text{ssg}.f, \text{ssg}.\text{outAdj})$ 
3:    $\text{from} \leftarrow \text{PATHSFROM}(\text{ssg}.i, \text{ssg}.f, \text{ssg}.\text{inAdj})$ 
4:    $\text{selected} \leftarrow \text{cmp}$ 
5:   for all  $p \in \text{inc}$  do
6:      $v \leftarrow \text{DELETELAST}(p)$ 
7:      $q \leftarrow \text{APPEND}(p, \text{CLONE}(\text{from}[v]))$ 
8:      $\text{selected} \leftarrow \text{selected} \cup \{q\}$ 
9:   return  $\text{selected}$ 

```

Dealing with updates: As discussed in §II-C, our TLA⁺ abstraction models only post and delete operations. We therefore insert put calls into the generated sequences by randomly augmenting the output of Algorithm 3 with zero to three consecutive put operations per created resource. This emulates common usage patterns (e.g., repeated profile edits), supports idempotency checks (identical consecutive updates with the same input), and exercises overwrite behaviour under successive updates with different inputs.

To keep sequences valid, updates must occur after creation and before deletion. Accordingly, we identify matching post/delete pairs and insert put operations between them. When a created resource has no corresponding delete in a sequence, we insert put operations at arbitrary positions, but always after the creation call.

Remarks: If there are parallel transitions, the algorithm may not achieve 100% transition coverage because each path is represented as a sequence of states, thereby losing information about which operation caused the transition. This representation choice is sufficient for full state coverage, but may lose transition identity when multiple operations connect the same abstract states; we quantify this effect in § IV-C. The assumption that there were no parallel transitions comes from the fact that in RESTful APIs, different operations typically have distinct preconditions or effects, so operations that appear to transition between the same abstract states usually differ in the data they manipulate (e.g., different resource IDs). The algorithm’s output is a collection of call sequences, i.e., invocations of the API operations, each consisting of an operation identifier and its parameters. Since these sequences are generated by TLC’s model checking, they contain model values rather than concrete API parameters.

III. ICEPICK

The ICEPICK tool comprises five core components that enable systematic black-box testing of RESTful APIs. The input generator produces random data conforming to the specification schemas. The state emulator maintains an abstract representation of the microservice state observed during testing, tracking which resources exist and their key properties. The evaluator checks whether GLACIER predicates hold at each execution point by interpreting contract clauses against observed responses and emulated state. The HTTP request handler manages all communication between the tester and the microservice, including request construction, transmission, and response capture. Finally, the tester orchestrates these components within the overall testing algorithm, coordinating the execution of call sequences and aggregating results.

A. Testing Algorithm

Algorithm 4 depicts ICEPICK’s testing procedure. It iterates over all generated call sequences, treating each as an independent test scenario. For each sequence, the algorithm resets the state emulator (line 4) to enforce isolation between runs and prevent state carryover from influencing subsequent sequences. It then executes each operation in sequence, aggregating the outcomes into the overall result.

Algorithm 4 ICEPICK’s testing algorithm.

```

1: function TEST( $\text{seqs}$ : call sequences,  $\text{spec}$ : extended spec)
2:    $\text{results} \leftarrow \emptyset$ 
3:   for all  $\text{seq} \in \text{seqs}$  do
4:      $\text{emulator}.\text{RESET}()$ 
5:     for all  $\text{op} \in \text{seq}$  do
6:        $\text{res} \leftarrow \text{TESTOPERATION}(\text{op}, \text{spec}, \text{ISLAST}(\text{op}))$ 
7:        $\text{results} \leftarrow \text{results} ++ \text{res}$ 
8:   return  $\text{results}$ 

```

Algorithm 5 presents ICEPICK’s algorithm for testing a single operation. It starts by verifying that system invariants hold in the pre-execution state (line 5). This check ensures

the system maintains global consistency properties before attempting the operation.

Algorithm 5 ICEPICK’s operation testing algorithm.

```

input:
  op: operation to test
  spec: GLACIER specification
  last: indicates if op is the last one in the call sequence
output:
  result: test outcome
1: function TESTOPERATION
2:   verb  $\leftarrow$  op.verb,      schema  $\leftarrow$  op.schema
3:   reqs  $\leftarrow$  op.reqs,    ens  $\leftarrow$  op.ens
4:   tlaID  $\leftarrow$  op.parameters, data  $\leftarrow$   $\emptyset$ 
5:   invs  $\leftarrow$  evaluator.EVALINV(spec.invs)
6:   switch verb
7:     case POST
8:       data  $\leftarrow$  generator.GENERATE(schema)
9:       pre  $\leftarrow$  evaluator.EVALPRE(reqs, data)
10:      resp  $\leftarrow$  handler.REQUEST(verb, data)
11:      if resp = OK then
12:        emulator.ADD(tlaID, data)
13:     case DELETE
14:       data  $\leftarrow$  emulator.RECYCLE(tlaID)
15:       if data =  $\emptyset$  then return NOT_TESTED
16:       pre  $\leftarrow$  evaluator.EVALPRE(reqs, data)
17:       resp  $\leftarrow$  handler.REQUEST(verb, data.id)
18:       if resp = OK then
19:         emulator.DELETE(tlaID, data)
20:     case PUT
21:       data  $\leftarrow$  emulator.RECYCLE(tlaID)
22:       if data =  $\emptyset$  then return NOT_TESTED
23:       data  $\leftarrow$  generator.GENERATE(schema)
24:       pre  $\leftarrow$  evaluator.EVALPRE(reqs, data)
25:       resp  $\leftarrow$  handler.REQUEST(verb, data.id)
26:       if resp = OK then
27:         emulator.UPDATE(tlaID, data)
28:       if resp = OK then
29:         pos  $\leftarrow$  evaluator.EVALPOS(ens, data, resp)
30:       if last then
31:         invs  $\leftarrow$  evaluator.EVALINV(spec.invs)
32:       return RESULT(resp, invs, pre, pos)

```

The algorithm then branches on the operation’s HTTP verb, handling data generation and recycling differently in each case: **POST operations** create new resources, so the input generator produces fresh data conforming to the operation’s request body schema (line 8). After evaluating preconditions using the generated data, the algorithm issues the HTTP request. If the service responds with a success code (2xx), the created resource is added to the state emulator (line 12), making it available for subsequent operations that may reference it.

DELETE operations remove existing resources, so the algorithm attempts to recycle previously generated data (line 14) from the state emulator, identified here by tlaID. If no matching data exists, typically because the corresponding post failed, the operation is marked as NOT_TESTED (line 15) and skipped. Otherwise, preconditions are evaluated, the delete request is issued with the resource identifier, and upon success, the resource is removed from the state emulator (line 19).

PUT operations update existing resources, requiring both

recycling of the target resource identifier (line 21) and generation of new data for the update payload. Like delete, if the required resource is unavailable in the state emulator, the operation is marked as NOT_TESTED (line 22). Otherwise, the algorithm generates new data conforming to the schema, evaluates the preconditions, issues the put request, and updates the emulated state upon success (line 27).

After executing the operation, if the service does not return a server error (as indicated by a 5xx response), the algorithm evaluates the postconditions using the operation’s ensures clauses. If the operation is the last in the sequence, the algorithm performs a final invariant check to ensure that system-wide properties still hold upon completion.

GET operations retrieve resources and are pure, i.e., they have no side-effects on the service’s state. These operations allow clients to inspect the system state and serve a dual purpose in ICEPICK. First, they enable evaluation of GLACIER contracts by providing observable state information for preconditions, postconditions, and invariants. Second, because get operations do not trigger state transitions in our TLA⁺ model, we do not test them as standalone operations. Instead, we use their responses as instrumentation and as the ground truth for validating the behaviour of state-mutating operations.

B. Result Classification

ICEPICK classifies each operation test into one of four categories: OK, WARN, ERR, or NOT_TESTED. This classification scheme provides feedback beyond simple pass-or-fail verdicts, reflecting the complexity of contract-based testing, where violations may be ambiguous.

NOT_TESTED indicates the state emulator lacks the required data to proceed with the request. This typically occurs when a delete or put operation references a resource that a post operation failed to create earlier in the sequence. For all other cases, Table I shows how the framework classifies operations based on three factors: whether preconditions held before execution, whether postconditions and invariants held after execution, and the HTTP response code returned by the service.

TABLE I
OPERATION RESULTS.

Preconditions	Postconditions	Invariants	Response Code	Result
T	T	T F	2xx / $\neg$ 2xx –	OK / ERR ERR
	F	T F	– 4xx / 2xx	ERR WARN / ERR
F	T	T F	– –	WARN ERR
	F	T F	4xx / $\neg$ 4xx 4xx / 2xx	OK / ERR OK / ERR

An operation test is classified as OK when the observed behaviour is consistent with the contract, i.e., preconditions, postconditions, and invariants hold, and the response code matches the outcome defined by the specification. The result of a test is ERR when there is a clear contract violation or an

TABLE II
SYSTEMS UNDER TEST DESCRIPTION.

System	Endpoints	Operations	Parameters	Schemas
Tournaments	11	19	15	4
Swagger-Petstore	13	19	17	6
E-Commerce	22	28	33	25
Features-Service	10	16	33	8

inconsistent response code. This includes several fault patterns. For instance, when preconditions hold, but postconditions or invariants do not, or when the response code indicates success while the contract evaluation indicates a failure. Finally, WARN handles ambiguous cases where the outcome is suspicious but not definitively wrong. Specifically, when preconditions hold, postconditions and invariants do not hold, and the service returned a 4xx status code. This is suspicious because the service rejected what appeared to be valid input per the specification, yet the contract violation suggests the system entered an unexpected state. In this same scenario, if the service replies with a 2xx response code, we are definitely dealing with an error, since the service claims success while violating the contract. In the latter warning scenario, the contract is violated, so by definition [31] we have no guarantee of the service’s state when the operation executes, and we conservatively classify the result as a warning.

This classification scheme enables ICEPICK to distinguish among failure types, allowing more effective debugging. The framework generates detailed test reports that identify the violated contracts, the sequence of operations leading to the failure, and the specific preconditions, postconditions, or invariants that were not met, providing developers with valuable diagnostic information.

IV. EVALUATION

Our experiments aim to evaluate both the framework and the proposed testing algorithm. Accordingly, this section is divided into three parts: SSG generation with TLC, call sequence generation, and testing, each with its own research questions.

All experiments were conducted on four identical Debian GNU/Linux 12 machines, each equipped with two AMD EPYC 7343 processors, for a total of 32 physical cores and 64 hardware threads, 128 GiB of DDR4 3200 MHz main memory, and a 450 GB SSD as the primary storage device. The machines were connected via a 2×10 Gbps network.

Table II describes the systems used to evaluate our framework. All services except the Tournaments System are drawn from the EMB [22]. Petstore [32] is the canonical example API for OAS; E-commerce service is a shopping cart application built as a technology-learning project [33]; and Features-Service manages configurable products and their features. For our experiments, we upgraded their OAS to version 3.1.0 and then validated and Dockerised each application for deployment.

A. Applicability Analysis – Requirements for effective use

ICEPICK is designed for testing systems that adhere to REST architectural principles and HTTP specifications. The approach works best when services properly implement standard HTTP semantics, including accurate status codes, resource-oriented design, and stateless interactions. These properties enable ICEPICK to effectively infer system state and detect invariant violations. During our evaluation, we found that many existing benchmarks used in prior work deviate significantly from these principles, making them unsuitable for demonstrating ICEPICK’s strengths. This observation highlights an important consideration for the research community: as REST testing techniques become more sophisticated, we need evaluation benchmarks that reflect proper REST architectural patterns. The EMB [22] represents a significant step forward, providing a valuable curated collection of systems for evaluation. However, our experience with two systems from this benchmark – Features-Service and E-Commerce API – shows that creating benchmarks comprehensively reflecting REST architectural principles is difficult, and even carefully curated collections may include systems with varying degrees of compliance.

When generating the extended specification files, we found several issues. Our generator (§II-B) invokes the official OAS validator, which revealed that several specifications were invalid. This allowed us to identify and correct issues such as missing required parameters, duplicated parameters, invalid URLs, undefined path parameters, invalid or unused schema definitions, missing response codes, and absent tag descriptions. While manually resolving most of these issues was straightforward, assigning meaningful tags required understanding the semantics of each operation and the resources it manipulates. We analysed the specification files and organised operations into groups (i.e., tags) based on their domain responsibilities and the resources they manage.

For Features-Service, we added tags Products, Features, Configurations, and Constraints aligned with the domain model and to separate concerns by resource. We also found signs that the API description is generated from source code: the file included an invalid schema name, Map<string, Link>, which we removed as an unused reference. We then inferred a TLA⁺ specification for this service from the OAS-generated contracts. However, testing revealed that all post operations lacked request-body definitions, which may be common practice, but does not follow the standard. RFC 9110 [34] (HTTP Semantics) specifies that a post is used for creating new resources that have yet to be identified by the origin server. Without request body schemas, clients cannot know what data is expected. This was compounded by Features-Service always returning success status codes 2xx, even for malformed or unprocessable requests. This violates basic HTTP principles – status codes should reflect the actual outcome of a request. With missing request-body definitions and success responses for everything, there is no way to distinguish valid operations from invalid ones, or to assess

whether the tool’s generated requests are actually correct.

For the E-Commerce API, our generator could not infer contracts because the API violates basic HTTP and REST conventions. For example, delete requests include a body (discouraged by RFC 9110) and their URIs do not conform to common design guidelines, which prevents us from deriving a meaningful TLA⁺ specification; therefore, we excluded them. Instead, we evaluate on Tournaments, Swagger-Petstore, and Features-Service, which better support the assessment of our approach.

Regarding the manual effort of writing TLA⁺ specifications, we can say both Tournaments and Petstore specifications are comparable in size. While we did not measure authoring time, the specifications appear to be small enough for a TLA⁺ practitioner to manage manually, though this remains to be validated empirically. Since reducing manual specification effort is a requirement for broader applicability, automating TLA⁺ generation from enriched OAS files is a primary direction for future work, as mentioned in § VII.

Overall, our applicability analysis indicates that ICEPICK is most effective for APIs that expose resource-oriented operations, provide accurate request/response schemas, and are compliant with HTTP status-code semantics.

B. SSG Generation

Next, we investigate the scalability of SSG generation using TLC and the relationship between model size and the resulting state space. We aim to determine whether TLC can generate SSGs for realistic API models within acceptable time and storage constraints. If model checking becomes impractical for model sizes beyond the trivial, the entire approach fails. As such, we address the following research question:

RQ1 What is the scalability limit on model size for which graph generation performance remains acceptable?

We ran TLC for the Tournaments System with configurations with total model sizes ranging from 3 to 12, e.g., 4 players, 4 tournaments, and 4 enrolments (p4.t4.e4). Figure 2 compares the model size with the number of states/transitions generated by the model checker. As expected, larger models yield more states, though models of the same size can differ by several orders of magnitude (e.g., p3.t3.e3 vs. p4.t4.e1). This is expected: fewer enrolments reduce the number of possible states by restricting the player/tournament combinations for each enrolment. We also observe that for relatively small models, e.g., p4.t3.e4, the SSG has more than 2 million states and 23 million transitions.

With state spaces reaching millions of states even for small models, one might consider leveraging TLC’s state-reduction techniques, such as symmetric model value sets. However, this approach does not apply here: in our examples, each model value maps directly to a concrete API resource, making the values intrinsically different rather than interchangeable.

Analysing TLC-generated SSG files, we observed substantial duplication in nodes and edges that would degrade call sequence generation performance, so we added a preprocessing step to remove redundancy. Table III reports size reduction.

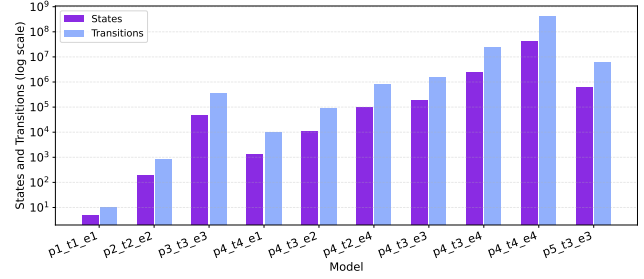


Fig. 2. Model size vs. number of states and transitions generated by TLC for the Tournaments API.

TABLE III
SSG GENERATION – EXECUTION TIME AND SIZE REDUCTION.

	Model	Exec. TLC	Original	Clean	Reduction
TOURNAMENTS	p1.t1.e1	< 01s	3 K	2 K	33.3 %
	p2.t2.e2	< 01s	357 K	166 K	53.5 %
	p3.t3.e3	00min 21s	243 M	62 M	74.5 %
	p4.t4.e1	< 01s	1.6 M	569 K	64.6 %
	p4.t3.e2	00min 05s	61 M	15 M	75.6 %
	p4.t2.e4	00min 45s	497 M	143 M	71.8 %
	p4.t3.e3	01min 49s	1.3 G	262 M	79.2 %
	p4.t3.e4	36min 09s	21 G	4 G	81.0 %
	p4.t4.e4	16h 16min	467 G	72 G	84.6 %
PETSTORE	p5.t3.e3	07min 48s	5.5 G	974 M	82.7 %
	u1.p1.o1	< 01s	5 K	3.5 K	30.0 %
	u2.p2.o2	< 01s	794 K	70 K	91.2 %
	u3.p3.o3	00min 05s	112 M	2.3 M	97.9 %
FEAT.	u4.p4.o4	11min 25s	15 G	95 M	99.4 %
	p1.c1.f1	< 01s	3.7 K	3.7 K	0 %
	p2.c2.f2	< 01s	1.3 M	710 K	45.4 %
	p3.c3.f3	06min 20s	5.2 G	1.9 G	63.6 %

Thanks to our pre-processing stage, the file-size reductions range from 30% to 99.4% across all 17 configurations; in over 75% of the cases, more than 50% of the original size is eliminated, and in three of the four Swagger-Petstore configurations, the reduction exceeds 90%. Pre-processing significantly reduces the size of the SSG representation, lowering storage costs and enabling faster subsequent traversals.

Experiments show that SSG generation scales to moderate model sizes: for the Tournaments API, configurations up to size 11 (≤ 21 GB) finish in under 40min, but size 12 balloons to 467 GB and 16h 16min; for Petstore, the largest tried (15 GB) finishes in 11min 25s. On our hardware, DOT/SSG outputs of roughly 5-21 GB can be produced within an hour, after which build time becomes impractical for quick testing.

RQ1 TLC scales to moderate model sizes – configurations within ~ 5 -21 GB complete within an hour. Beyond this (e.g., 467 GB for Tournaments with 12 model values), generation becomes impractical (16+ hours).

C. Call Sequence Generation

This section evaluates the scalability and effectiveness of our Java implementation of the call-sequence generation algo-

rithm for SSG exploration. Even if TLC successfully generates an SSG, the call sequence generation algorithm must be able to process it within available memory; understanding these limits prevents users from attempting infeasible configurations. We configured the JVM with 100 GB heap and ran the algorithm for each model configuration, addressing the following research questions:

RQ2 How large can the SSG grow while our algorithm still runs without exhausting our memory limits?

RQ3 What configurations should we test, considering our time and memory constraints?

We consider RQ1, RQ2, and RQ3 to establish ICEPICK’s practical feasibility by determining: (i) whether TLC can generate SSGs within acceptable time/storage limits; (ii) whether the call sequence algorithm can process these graphs within memory constraints; and (iii) which model configurations are tractable for real-world testing. Together, they bridge the gap between theoretical soundness and practical applicability.

Table IV summarises the results of our experiments per model, reporting the resulting SSG’s size (in terms of number of states and transitions), and the number of distinct paths generated. We also show the graph coverage obtained from the generated path set, in terms of both visited states and visited transitions (full results in the replication package [23]). We only execute tests for one- and two-value-per-resource configurations, since three values yield an infeasible number of call sequences (tens of thousands), which translates into millions of API requests.

For all SSGs, the algorithm quickly achieves full state coverage, effectively enumerating all possible paths. We were not able to generate call sequences for the p3.c3.f3 of Features-Service (FS.) on our setup due to memory constraints. Petstore (Pets.) configurations achieve only 74% to 84% transition coverage, unlike the 100% obtained for Tournaments (Tour.) and Features-Service. This is due to the parallel-transition assumption discussed in §II-D: some Petstore operations share the same abstract states but differ in data manipulation (e.g., createUser and createUserWithList both transition from a state where the user does not exist to one where it does). Since paths are represented only as state sequences, the algorithm selects one transition per state pair and misses the other.

Regarding SSG size limits, the largest configuration we could run within our memory constraints was p3.t3.e3 for the Tournaments system, with 9 model values, 46K states, and 349K transitions.

RQ2 With a 100 GB heap, the algorithm handles SSGs with up to ~46K states and ~349K transitions.

RQ3 Only one- and two-value-per-resource configurations are practical; three yield tens of thousands of sequences (millions of API requests), making testing infeasible.

D. Testing

In this section, we investigate ICEPICK’s ability to uncover faults through systematic state-space exploration, and assess

TABLE IV
CALL SEQUENCE GENERATION.

	Model	Graph Size		Path Size			Graph Cov.		
		States	Trans.	Min	Max	Avg	States	Trans.	Paths
TOUR.	p1.t1.e1	6	10	6	11	9	100%	100%	7
	p2.t2.e2	193	872	9	27	17	100%	100%	721
	p3.t3.e3	46K	349K	9	39	24	100%	100%	312K
PETS.	u1.p1.o1	7	19	3	6	4	100%	84.2%	15
	u2.p2.o2	73	574	5	10	7	100%	81.2%	507
	u3.p3.o3	1361	19.9K	7	13	9	100%	74.4%	18.5K
FS.	p1.c1.f1	12	36	2	7	5	100%	100%	32
	p2.c2.f2	4490	49.8K	6	16	10	100%	100%	46K

TABLE V
TEST RESULTS.

	Model	Exec. Time	WARN	ERR	NOT_TESTED
TOUR.	p1.t1.e1	00min 02s	0	0	0
	p2.t2.e2	02min 20s	0	0	0
	p1.t1.e1.ext	00min 14s	0	0	0
T. ERR	p1.t1.e1.ERR	00min 02s	8	5	0
	p2.t2.e2.ERR	02min 44s	1658	459	49
	p1.t1.e1.ERR.ext	00min 17s	7	5	0
PETS.	u1.p1.o1	00min 01s	0	0	3
	u2.p2.o2	00min 11s	0	0	72
FS.	p1.c1.f1	< 01s	0	97	50

whether enriching automatically generated contracts with manually specified semantic properties enhances fault detection beyond basic HTTP semantics. As such, we aim to answer the following research questions:

RQ4 How effective is ICEPICK in finding faults?

RQ5 Does the manual extension of specifications result in an increased number of detected faults?

To evaluate fault-detection, we injected three faults into the Tournaments system: (i) deletePlayer does not remove the player; (ii) deleteTournament removes a random tournament; and (iii) deleteEnrolment fails to update the Tournament’s players collection, violating referential integrity. The latter was discovered during preliminary testing rather than intentionally injected.

Table V shows the test execution results for the Tournaments, Petstore, Features-Service, and the faulty Tournaments system, across different model configurations. Columns WARN and ERR count the number of individual operation executions that were classified as errors or warnings, according to ICEPICK’s result classification scheme in Table I. Each operation tested during the execution of all call sequences contributes one result, which is categorised as OK, WARN, ERR, or NOT_TESTED. So, e.g., the 97 errors of Features-Service imply 97 operation executions violated their contracts or exhibit inconsistent behaviour.

All Tournaments configurations completed successfully (under 3 minutes). For Petstore, both completed without warnings or errors, but left 3 and 72 operations untested:

instances of `deleteUser` in sequences where users were created via `createUserWithList` rather than `postUser`. Since `createUserWithList` stores users as an array while `deleteUser` expects a single object, a distinction not inferable from the specification, ICEPICK marks these as NOT_TESTED.

Features-Service was tested with a single configuration due to its non-compliance with REST architectural principles and HTTP semantics. ICEPICK found 97 errors and 50 untested operations in the smallest possible configuration for this service, which completed in under one second. All operations marked as errors are post, and all operations marked as NOT_TESTED are operations depending on resource creation (e.g., ICEPICK cannot test the deletion of a product if the product could not be added to the system). These operations are classified as errors because they lack definitions in their request bodies.

ICEPICK detected all injected faults in the Tournaments system (T. ERR), even in the smallest configuration. However, some faults were semantically subtle. A representative example is the referential-integrity fault in `deleteEnrolment`. ICEPICK first creates a state in which a player is enrolled in a tournament, then executes the deletion and checks the resulting state against the specification. Although the faulty implementation returns request bodies lacking information and produces the same HTTP response as the correct one, it leaves the tournament's player collection unchanged, resulting in an inconsistent state where the enrolment is removed, but the tournament still references the player. This shows why stateful, model-guided exploration is necessary: the fault only becomes visible after establishing the relevant multi-resource state.

The extended specification version shows comparable detection capability to the generated version for the small model, suggesting that even automatically generated contracts are effective at catching the injected defects. However, the extended specification produced one fewer warning than the generated one. Log analysis revealed a `postEnrolment` operation failed both its pre and postconditions in the extended specification test, resulting in an OK classification when the service returned a 4xx response (according to Table I). The postcondition failure was due to tournament capacity: random data generation created a tournament with a small capacity that was exhausted by previous enrolments. In contrast, the baseline test randomly generated a tournament with a larger capacity, so the postcondition passed, resulting in a WARN classification for the same test.

ICEPICK proved effective at finding faults. It detected all three injected faults in the Tournaments system, even when using the smallest model configuration. It also uncovered 97 errors in Features-Service, all related to missing request body definitions and violations of HTTP semantics. However, ICEPICK's effectiveness depends on how well the API adheres to REST principles: well-designed APIs, such as Petstore and Tournaments, enable comprehensive testing, whereas poorly compliant systems, such as Features-Service, immediately reveal structural problems that block deeper exploration. In essence, ICEPICK was effective in two different ways: it detected injected behavioural faults in the

REST-compliant Tournaments system, and it exposed severe specification/HTTP-semantics deficiencies in Features-Service that prevented meaningful deeper exploration.

Manually extending specifications did not substantially improve fault detection over automatically generated contracts in our injected-fault setting: both detected all Tournaments faults, and the only observed difference was a warning caused by random data generation. This indicates that CRUD- and HTTP-based inferred contracts were sufficient for the fault types represented in our injections. However, this should not be interpreted as evidence that manual contracts are unnecessary: manually added GLACIER predicates are expected to be more beneficial for domain-specific semantic faults that are not captured by generic CRUD behaviour and were not exercised by our injected faults.

RQ4 ICEPICK detected all injected Tournaments faults and 97 errors in Features-Service, though its effectiveness depends on REST compliance.

RQ5 In our injected-fault setting, manual extensions did not improve detection; inferred contracts sufficed for the exercised fault patterns.

V. THREATS TO VALIDITY

Internal Validity. The correctness and completeness of the generated GLACIER contracts pose a threat: if a predicate is incorrect, or a contract is overly permissive or restrictive, the observed effectiveness may reflect specification quality rather than ICEPICK's fault-finding capability – i.e., overconstrained specifications may lead to missing the exploration of valid call sequences, while underconstrained ones may lead to the generation of invalid call sequences [35]. To mitigate this we manually reviewed all generated contracts, checking them against the OAS textual descriptions and CRUD operation semantics. Similarly, TLA⁺ specifications may diverge from the real system due to abstraction, so we sanity-checked all cases and iteratively refined them whenever observed API behaviour differed from SSG-implied behaviour. Moreover, our injected faults primarily target behaviours that can be expressed by inferred CRUD- and HTTP-level contracts, potentially understating the added value of manually specified domain-specific predicates. A further internal validity threat is the framework's implementation: defects in any component could bias results. While we tested all components, we cannot guarantee the absence of faults. To mitigate this and support reproducibility, we publicly release the framework implementation and the case studies in our replication package [23].

External Validity. We evaluated ICEPICK on only three systems due to the scarcity of high-quality open-source APIs. To mitigate this, we plan to evaluate ICEPICK with more EMB [22] systems. Finally, the OAS files we used may be cleaner and more complete than real-world specifications. In practice, specifications can be outdated – i.e., the specification file does not evolve at the same pace as the implementation – or inconsistent, which can affect the framework's effectiveness.

VI. RELATED WORK

TLA⁺ has seen significant industrial adoption beyond Amazon [36], including use at MongoDB for verifying distributed reconfiguration protocols [37]. While these applications focus on verifying distributed system designs, our work adapts TLA⁺ to black-box API testing, a challenging problem requiring operational dependencies inference, generation of schema-compliant inputs, and the definition of test oracles beyond HTTP status codes. While existing methods address these issues to varying degrees, key gaps remain; this work addresses them by combining contract-based verification with advanced call sequence generation.

Several tools infer operation dependencies to generate valid call sequences. RESTTESTGEN [8] builds a static operation dependency graph (ODG) from inter-operation data flows; MoREST [9] extends this with a dynamic, runtime-updated resource property graph (RPG) to enable longer sequences, but requires exploratory calls; RESTler [10] combines fuzzing with CRUD semantics and feedback from prior requests to incrementally infer dependencies. JepREST [38] targets concurrency correctness by verifying linearisability of REST operations under concurrent execution, complementing ICEPICK's focus on sequential state-space coverage.

Beyond operation dependencies, some tools focus on parameter dependencies. RESTest [11] analyses inter-parameter relationships to construct valid calls and produce both nominal and faulty tests. It uses richer oracles beyond 5xx status codes by also checking OAS conformance and whether valid inputs receive successful responses. RestCT [12] applies combinatorial testing across operations and parameters, generating operation sequences with constrained sequence covering arrays and input assignments with classical covering arrays. It uses CRUD semantics and URI hierarchy to handle inter-operation constraints. RestCT provides systematic t-way coverage of operation combinations, i.e., every combination of t operations (or t parameter values) appears at least once across the generated test sequences, which fundamentally differs from our state-space coverage over the formal behavioural model.

EvoMaster [5], [7] treats test generation as a search-based optimisation problem, using the Many Independent Objective (MIO) algorithm to evolve tests that maximise coverage and fault detection. It features two execution modes: the white-box mode [5] instruments the SUT collecting coverage metrics to guide the search; in the black-box mode [7], it relies on CRUD semantics and URI hierarchy to structure test sequences.

ModelFuzz [39] uses TLA⁺ to guide fuzzing of distributed systems implementations. Instead of exploring program input spaces as traditional fuzzers do, it explores event schedules in distributed systems, using abstract model state coverage as feedback to guide test generation. This approach maps implementation events to model actions and uses a controlled model checker to track which abstract states have been covered, and to mutate schedules that reach new states. While ModelFuzz is designed for distributed system testing rather than RESTful APIs, it shares the underlying philosophy of leveraging higher-

level abstractions to define test oracles and to guide exploration beyond output validation.

Existing tools often misclassify 5xx responses as defects, generate mostly random inputs, and lack robust mechanisms to verify behaviour beyond status codes. These approaches either improve dependency inference, coverage, or input generation, but still rely on limited oracles and do not support systematic state-space exploration. The literature also includes additional approaches, such as specification-based techniques [14], metamorphic testing strategies [15], and the reinforcement learning-based exploration proposed by ARAT-RL [40]. Although these methods address related aspects of API testing, they target different problem settings and rely on testing paradigms that differ substantially from our approach.

VII. CONCLUSION AND FUTURE WORK

This paper presents ICEPICK, a framework for systematic black-box testing of RESTful APIs using model checking to achieve comprehensive state-space coverage. ICEPICK combines TLA⁺ specifications with the TLC model checker to generate test suites that provably cover the modelled behavioural space. It derives call sequences through a coverage-guided BFS traversal guaranteeing full state coverage and, when the abstraction has no parallel transitions, full transition coverage. Our evaluation highlights both the potential and the boundaries of the approach. For REST-compliant APIs, ICEPICK uncovers subtle multi-operation faults (e.g., referential integrity violations) that tools relying solely on HTTP status code oracles would miss. However, REST compliance is not just a desirable property but a hard requirement: as our experience with Features-Service and E-Commerce APIs shows, when APIs deviate from REST principles, ICEPICK may be unable to infer contracts, making testing infeasible before it begins. When testing is feasible, the framework produces reproducible test reports – an advantage over search-based or fuzzing approaches whose outputs vary across runs. At the same time, our results clarify where ICEPICK's current limits lie. The approach requires APIs to adhere to REST principles, manual effort to write TLA⁺ specifications, and careful model sizing to stay within tractable time and memory bounds. These are not fundamental barriers, but they do define the conditions under which ICEPICK is most effective.

Future Work. Several directions are worth exploring. First, our evaluation only covers three systems, so testing EMB systems would strengthen the external validity of our findings. Second, writing TLA⁺ specifications currently requires manual effort and formal methods expertise; automating this step to derive the full TLA⁺ specification directly from the extended file with GLACIER contracts is a primary direction for future work. Third, adapting ICEPICK to handle partially REST-compliant APIs, by relaxing assumptions about status code semantics or resource-oriented URI design, would broaden its applicability to real-world systems. Finally, when SSG contains parallel transitions, our algorithm cannot guarantee full transition coverage; extending the call-sequence generation to track transition identity, rather than representing paths purely

as state sequences, would close this gap while preserving strong coverage guarantees.

Overall, ICEPICK shows that combining model checking with automated contract inference and executable test oracles can generate comprehensive, reproducible API test suites with strong coverage guarantees.

ACKNOWLEDGMENTS

The authors were funded by NOVA LINCS (UID/04516/2025), FCT I.P., and the EU Horizon TaRDIS project (Grant Agreement No. 101093006).

REFERENCES

- [1] E. M. Clarke, "Model checking," in *International conference on foundations of software technology and theoretical computer science*. Springer, 1997, pp. 54–56.
- [2] P. Godefroid and K. Sen, *Combining Model Checking and Testing*. Cham: Springer International Publishing, 2018.
- [3] P. E. Ammann, P. E. Black, and W. Majurski, "Using model checking to generate tests from specifications," in *Proceedings second international conference on formal engineering methods (Cat. No. 98EX241)*. IEEE, 1998, pp. 46–54.
- [4] M. Fowler and J. Lewis, "Microservices," <http://martinfowler.com/articles/microservices.html>, 2014, accessed in January 2024.
- [5] A. Arcuri, "RESTful API automated test case generation with evomaster," *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 1, 2019.
- [6] Swagger, "OpenAPI specification," <https://swagger.io/specification/>, 2021, accessed in January 2024.
- [7] A. Arcuri, "Automated black- and white-box testing of RESTful APIs with EvoMaster," *IEEE Software*, vol. 38, no. 3, pp. 72–78, 2021.
- [8] E. Viglianisi, M. Dallago, and M. Ceccato, "RESTTESTGEN: Automated black-box testing of RESTful APIs," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 2020, pp. 142–152.
- [9] Y. Liu, Y. Li, G. Deng, Y. Liu, R. Wan, R. Wu, D. Ji, S. Xu, and M. Bao, "Morest: Model-based RESTful API testing with execution feedback," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. Association for Computing Machinery, 2022, p. 1406–1417.
- [10] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API fuzzing," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 748–758.
- [11] A. Martín-López, S. Segura, and A. Ruiz-Cortés, "Restest: automated black-box testing of RESTful web APIs," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 682–685. [Online]. Available: <https://doi.org/10.1145/3460319.3469082>
- [12] H. Wu, L. Xu, X. Niu, and C. Nie, "Combinatorial testing of RESTful APIs," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 426–437.
- [13] J. C. Alonso, S. Segura, and A. Ruiz-Cortés, "AGORA: Automated generation of test oracles for REST APIs," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 1018–1030.
- [14] H. Ed-douibi, J. L. Cánovas Izquierdo, and J. Cabot, "Automatic generation of test cases for REST APIs: A specification-based approach," in *2018 IEEE 22nd International Enterprise Distributed Object Computing Conference (EDOC)*, 2018, pp. 181–190.
- [15] S. Segura, J. A. Parejo, J. Troya, and A. Ruiz-Cortés, "Metamorphic testing of RESTful Web APIs," in *Proceedings of the 40th International Conference on Software Engineering*, ser. ICSE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 882.
- [16] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.
- [17] L. Lamport, "The temporal logic of actions," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 16, no. 3, pp. 872–923, 1994.
- [18] —, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, June 2002. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/specifying-systems-the-tla-language-and-tools-for-hardware-and-software-engineers/>
- [19] Y. Yu, P. Manolios, and L. Lamport, "Model checking TLA+ specifications," in *Advanced research working conference on correct hardware design and verification methods*. Springer, 1999, pp. 54–66.
- [20] M. A. Kuppe, L. Lamport, and D. Ricketts, "TLA+ model checking made symbolic," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, vol. 3, no. OOPSLA, 2019, pp. 123:1–123:30.
- [21] A. C. Ribeiro, "Glacier generator & catalogue," <https://github.com/acm-ribeiro/glacier-generator>, 2025.
- [22] A. Arcuri, M. Zhang, A. Golmohammadi, A. Belhadi, J. P. Galeotti, B. Marculescu, and S. Seran, "Emb: A curated corpus of web/enterprise applications and library support for software testing research," in *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2023, pp. 433–442.
- [23] A. Ribeiro, M. Mamede, and C. Ferreira, "Replication package for systematic API testing through model checking and executable contracts," <https://doi.org/10.5281/zenodo.19404322>, Dec. 2025.
- [24] A. C. Ribeiro, "Invariant-driven automated testing," M.S. thesis, NOVA School of Science and Technology, February 2021.
- [25] R. J. Dias, C. Ferreira, J. Fiedor, J. M. Lourenço, A. Smrcka, D. G. Sousa, and T. Vojnar, "Verifying concurrent programs using contracts," in *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2017, pp. 196–206.
- [26] R. Heckel and M. Lohmann, "Towards contract-based testing of web services," *Electronic Notes in Theoretical Computer Science*, vol. 116, pp. 145–156, 2005.
- [27] G. Dai, X. Bai, Y. Wang, and F. Dai, "Contract-based testing for web services," in *31st Annual International Computer Software and Applications Conference (COMPSAC 2007)*, vol. 1, 2007, pp. 517–526.
- [28] B. K. Aichernig, *Contract-Based Testing*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 34–48. [Online]. Available: https://doi.org/10.1007/978-3-540-40007-3_3
- [29] D. Wang, W. Dou, Y. Gao, C. Wu, J. Wei, and T. Huang, "Model checking guided testing for distributed systems," in *Proceedings of the Eighteenth European Conference on Computer Systems*, ser. EuroSys '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 127–143. [Online]. Available: <https://doi.org/10.1145/3552326.35587442>
- [30] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.
- [31] B. Meyer, "Applying 'design by contract'," *Computer*, vol. 25, no. 10, pp. 40–51, 1992.
- [32] Swagger, "Petstore - openapi 3.0," <https://petstore3.swagger.io/>, 2018, accessed in September 2025.
- [33] O. Sahin, M. Zhang, and A. Arcuri, "WFC/WFD: Web fuzzing commons, dataset and guidelines to support experimentation in REST API fuzzing," 2025. [Online]. Available: <https://arxiv.org/abs/2509.01612>
- [34] J. R. Roy T. Fielding, Mark Nottingham, "HTTP semantics (POST)," <https://httpwg.org/specs/rfc9110.html#POST>, RFC 9110, 6 2022, accessed in December 2025.
- [35] M. Politano, V. Bengolea, F. Molina, N. Aguirre, M. F. Frias, and P. Ponzio, "Efficient bounded exhaustive input generation from program apis," in *International Conference on Fundamental Approaches to Software Engineering*. Springer, 2023, pp. 111–132.
- [36] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardouff, "How Amazon Web Services uses formal methods," *Communications of the ACM*, vol. 58, no. 4, pp. 66–73, 2015.
- [37] W. Schultz, I. Dardik, and S. Tripakis, "Formal verification of a distributed dynamic reconfiguration protocol," in *International Conference on Principles of Distributed Systems (OPODIS)*, 2019.
- [38] S. Simoes, A. Ribeiro, C. Ferreira, and N. Preguica, "JepREST: Functional tests for distributed REST applications," 2023. [Online]. Available: <https://arxiv.org/abs/2303.14104>
- [39] E. B. Gulcan, B. K. Ozkan, R. Majumdar, and S. Nagendra, "Model-guided fuzzing of distributed systems," *Proc. ACM Program. Lang.*, vol. 9, no. OOPSLA2, 2025.
- [40] M. Kim, S. Sinha, and A. Orso, "Adaptive REST API testing with reinforcement learning," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 446–458.